Fall 2025 CPTS530 Midterm Project
MATH 448-548
Numerical Analysis

Aryan Ritwajeet Jha

14 October 2025

Contents

Problem 1 **2**

Problem 2 **3**

Problem 3 **5**

Problem 4 **9**
Problem 4.1 9
Problem 4.2 12
Problem 4.3 14
Problem 4.4 16
Problem 4.5 17
Problem 4.6 19

Problem 5 **20**
Problem 5.1 20
Problem 5.2 22
Problem 5.3 24
Problem 5.4 26

A Code Implementations **32**
A.1 Problem 3: LU and Cholesky Factorization 32
A.2 Problem 5-3: Ridders' Method Function 35

Problem 1

Problem Statement: Use Taylor theorem to derive the error term for the approximate formula

$$f'(x) \approx \frac{1}{2h}(-3f(x) + 4f(x+h) - f(x+2h)).$$

Solution:

We are given the finite-difference formula:

$$f'(x) \approx \frac{1}{2h}\left[-3f(x) + 4f(x+h) - f(x+2h)\right]$$

Expand $f(x+h)$ and $f(x+2h)$ about x using Taylor's theorem, keeping terms up to h^2 and representing higher-order terms as k_1h^3 and k_2h^4:

$$f(x+h) = f(x) + hf'(x) + \frac{h^2}{2}f''(x) + k_1h^3$$
$$f(x+2h) = f(x) + 2hf'(x) + 2h^2f''(x) + k_2h^3$$

where k_1, k_2 are finite constants depending on third order derivatives of f at x.

Substitute into the formula:

$$\begin{aligned}
&\frac{1}{2h}\left[-3f(x) + 4f(x+h) - f(x+2h)\right] \\
&= \frac{1}{2h}\Big[-3f(x) + 4\left(f(x) + hf'(x) + \frac{h^2}{2}f''(x) + k_1h^3\right) \\
&\quad - \left(f(x) + 2hf'(x) + 2h^2f''(x) + k_2h^3\right)\Big]
\end{aligned}$$

Simplifying the terms in the square brackets:

$$= \frac{1}{2h}\Big[2hf'(x) + (4k_1 - k_2)h^3\Big]$$

Now, divide each term by $2h$:

$$= f'(x) + \frac{1}{2}(4k_1 - k_2)h^2$$

Thus, the leading error term is $O(h^2)$:

$$\text{error} \sim O(h^2)$$

This shows the method is second-order accurate. Note that I've already checked if the third order term vanishes if we write out the taylor's polynomial to the third order, and it does not.

Also note that without knowledge of the specific function f or f''' to be precise, I cannot determine the exact constants in the error term, only confirm the order of accuracy.

Problem 2

Problem Statement: Consider the following variation of the Newton's method:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_0)}.$$

Find constants C and s such that

$$e_{n+1} = Ce_n^s.$$

Solution:
We start with the given iteration formula:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_0)}$$

Let the true root be r, and define the error at step n as $e_n = x_n - r$. Then,

$$\begin{aligned} e_{n+1} &= x_{n+1} - r \\ &= x_n - \frac{f(x_n)}{f'(x_0)} - r \\ &= (x_n - r) - \frac{f(x_n)}{f'(x_0)} \\ &= e_n - \frac{f(x_n)}{f'(x_0)} \end{aligned}$$

Next, we use the Taylor expansion of $f(x_n)$ around the root r:

$$f(x_n) = f(r) + f'(r)(x_n - r) + \frac{f''(r)}{2}(x_n - r)^2 + O((x_n - r)^3)$$

Substituting this expansion into our expression for e_{n+1} gives:

$$\begin{aligned} e_{n+1} &= e_n - \frac{f(r) + f'(r)e_n + \frac{f''(r)}{2}e_n^2 + O(e_n^3)}{f'(x_0)} \\ &= e_n - \frac{f(r)}{f'(x_0)} - \frac{f'(r)e_n}{f'(x_0)} - \frac{f''(r)}{2f'(x_0)}e_n^2 - \frac{O(e_n^3)}{f'(x_0)} \end{aligned}$$

Rearranging terms, we have:

$$e_{n+1} = \left(1 - \frac{f'(r)}{f'(x_0)}\right) e_n - \frac{f''(r)}{2f'(x_0)}e_n^2 + O(e_n^3)$$

To determine the order of convergence, we analyze the leading term in the expression for e_{n+1}. Assuming that $f'(r) \neq f'(x_0)$, the term $\left(1 - \frac{f'(r)}{f'(x_0)}\right) e_n$ dominates as $e_n \to 0$. Thus, we identify:

$$C = \left|1 - \frac{f'(r)}{f'(x_0)}\right|, \quad s = 1$$

Therefore, the error satisfies:

$$e_{n+1} = Ce_n^s$$

with $C = \left|1 - \frac{f'(r)}{f'(x_0)}\right|$ and $s = 1$. It may be noted that if $f'(r) = f'(x_0)$, the leading term vanishes, and the convergence order would then be determined by the next term(s), depending on the first non-vanishing derivative of f at the root r, so if $f''(r) \neq 0$, we would have quadratic convergence ($s = 2$), and if $f''(r) = 0$ but $f'''(r) \neq 0$, we would have cubic convergence ($s = 3$), and so on.

If $f'(r) \neq f'(x_0)$, we don't really have any idea whether the method converges or not; while the error ratios are linear, we cannot guarantee $0 < C < 1$; if $C \geq 1$, the method may very well diverge. In fact if $f'(x_0)$ and $f'(r)$ have opposite signs, then surely $C > 1$ and the method diverges.

The following **box** summarizes the error order parameters and remarks on convergence for this fixed-point iteration method:

Convergence Summary

Case-by-case conclusion:

- If $f'(x_0) \neq f'(r)$:

$$s = 1, \qquad C = \left|1 - \frac{f'(r)}{f'(x_0)}\right|, \qquad \text{Convergence not guaranteed}$$

- If $f'(x_0) = f'(r)$ and $f^{(k)}(r) = 0$ for $2 \leq k < n$, but $f^{(n)}(r) \neq 0$:

$$s = n, \qquad C = \left|\frac{f^{(n)}(r)}{n!\, f'(x_0)}\right|, \qquad \text{Converges with order } n$$

(where n is the smallest integer ≥ 2 such that $f^{(n)}(r) \neq 0$)

Problem 3

Problem Statement: Consider the linear system $\mathbf{Ax} = \mathbf{b}$, where

$$\mathbf{A} = \begin{bmatrix} 6.25 & -1 & 0.5 \\ -1 & 5 & 2.12 \\ 0.5 & 2.12 & 3.6 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} 7.5 \\ -8.68 \\ -0.24 \end{bmatrix}.$$

Write a computer program for LU-factorization with a unit lower triangular $\mathbf{L}$ (meaning that the diagonal entries should be equal to one). Then write a program for the Cholesky factorization.

WARNING: avoid using shortcuts. The programming should be done "from scratch".

Solution:

Implementation Approach

I implemented both LU (Doolittle) and Cholesky factorizations from scratch in Julia, using simple nested loops without any built-in matrix factorization functions. The implementations follow the standard algorithms.

Method 1: LU Factorization

The LU factorization decomposes $\mathbf{A} = \mathbf{LU}$, where $\mathbf{L}$ is unit lower triangular and $\mathbf{U}$ is upper triangular. Figure 1 shows the complete output including the factorization, verification, and solution.

```
========================================
METHOD 1: LU Factorization
========================================

L (lower triangular, 1's on diagonal):
3×3 Matrix{Float64}:
  1.0    0.0       0.0
 -0.16   1.0       0.0
  0.08   0.454545  1.0

U (upper triangular):
3×3 Matrix{Float64}:
 6.25  -1.0   0.5
 0.0    4.84  2.2
 0.0    0.0   2.56

Verification: max|A - LU| = 4.441e-16

Solution from LU factorization:
x =
3-element Vector{Float64}:
  0.8
 -2.0
  1.0
Residual: max|Ax - b| = 2.220e-16
Difference from Julia's A\b: 2.220e-16
✓ LU solution matches built-in solver!
```

Figure 1: LU Factorization output showing the factored matrices $\mathbf{L}$ and $\mathbf{U}$, verification that $\mathbf{A} = \mathbf{LU}$, the computed solution $\mathbf{x}$, and comparison with Julia's built-in solver. The factorization is verified to machine precision ($\sim 10^{-16}$), and the solution matches the built-in solver exactly.

The factorization yields:

$$\mathbf{L} = \begin{bmatrix} 1.0 & 0.0 & 0.0 \\ -0.16 & 1.0 & 0.0 \\ 0.08 & 0.454545 & 1.0 \end{bmatrix}, \quad \mathbf{U} = \begin{bmatrix} 6.25 & -1.0 & 0.5 \\ 0.0 & 4.84 & 2.2 \\ 0.0 & 0.0 & 2.56 \end{bmatrix}$$

Verification: $\max|\mathbf{A} - \mathbf{LU}| = 4.441 \times 10^{-16}$

Method 2: Cholesky Factorization

The Cholesky factorization exploits the symmetry and positive-definiteness of $\mathbf{A}$, decomposing it as $\mathbf{A} = \mathbf{L}\mathbf{L}^T$, where $\mathbf{L}$ is lower triangular. Figure 2 shows the complete output.

```
================================================
METHOD 2: Cholesky Factorization
================================================

L (Cholesky factor):
3×3 Matrix{Float64}:
  2.5  0.0  0.0
 -0.4  2.2  0.0
  0.2  1.0  1.6

Verification: max|A - LL'| = 8.882e-16

Solution from Cholesky factorization:
x =
3-element Vector{Float64}:
  0.8
 -1.9999999999999996
  0.9999999999999998
Residual: max|Ax - b| = 2.220e-16
Difference from Julia's A\b: 4.441e-16
✓ Cholesky solution matches built-in solver!

Comparison: max|x_LU - x_Cholesky| = 4.441e-16
✓ Both methods agree perfectly!
```

Figure 2: Cholesky Factorization output showing the Cholesky factor $\mathbf{L}$, verification that $\mathbf{A} = \mathbf{L}\mathbf{L}^T$, the computed solution $\mathbf{x}$, and comparison with Julia's built-in solver. The factorization is verified to machine precision, and both LU and Cholesky methods produce identical solutions.

The Cholesky factor is:

$$\mathbf{L}_{\text{Cholesky}} = \begin{bmatrix} 2.5 & 0.0 & 0.0 \\ -0.4 & 2.2 & 0.0 \\ 0.2 & 1.0 & 1.6 \end{bmatrix}$$

Verification: $\max|\mathbf{A} - \mathbf{L}\mathbf{L}^T| = 8.882 \times 10^{-16}$

Final Solution

Both methods yield the same solution (verified to machine precision):

$$\boxed{\mathbf{x} = \begin{bmatrix} 0.8 \\ -2.0 \\ 1.0 \end{bmatrix}}$$

Verification and Testing

The implementations were thoroughly verified:

- LU factorization accuracy: $\max|\mathbf{A} - \mathbf{LU}| = 4.441 \times 10^{-16}$
- Cholesky factorization accuracy: $\max|\mathbf{A} - \mathbf{L}\mathbf{L}^T| = 8.882 \times 10^{-16}$
- Solution residual: $\max|\mathbf{Ax} - \mathbf{b}| = 2.220 \times 10^{-16}$
- Comparison with Julia's built-in solver: $\max|\mathbf{x}_{\text{computed}} - \mathbf{x}_{\text{builtin}}| = 2.220 \times 10^{-16}$
- LU vs Cholesky solution: $\max|\mathbf{x}_{\text{LU}} - \mathbf{x}_{\text{Cholesky}}| = 4.441 \times 10^{-16}$

All errors are at machine precision level ($\sim 10^{-16}$), confirming the correctness of both implementations. The complete Julia codebase can be viewed in Appendix A.1.

Problem 4

Problem Statement: Consider the equation

$$f(x) = 0,$$

where

$$f(x) = x - \frac{1}{2}\left(\cos(x/2) - \left|x - \frac{1}{2}\right|\right).$$

Do the following.

Part 1

Rewrite the equation as a fixed point problem for a function $T(x)$ related to $f(x)$. Prove that $T(x)$ (i) maps the interval $[0.45, 0.55]$ into itself, and (ii) $T(x)$ is contractive on this interval. Based on the theorems in the books and notes, make conclusions about existence and uniqueness of a solution to $f(x) = 0$.

Solution:

For determining a suitable fixed point function $T(x)$, we start with the given equation:

$$f(x) = 0 \implies x = \frac{1}{2}\left(\cos(x/2) - \left|x - \frac{1}{2}\right|\right).$$

Thus, we can define the fixed point function as:

$$T(x) = \frac{1}{2}\left(\cos(x/2) - \left|x - \frac{1}{2}\right|\right).$$

To get an idea of the behaviours of functions $f(x)$ and $T(x)$, I plotted both in desmos, as shown in Figure 3 and Figure 4.

In Figure 4, I can see that $T(x)$ appears to map the interval $[0.45, 0.55]$ into itself. As for locating the fixed point(s), I can see that $T(x)$ intersects the line $y = x$ at a single point in this interval, suggesting a unique fixed point. The plot of $f(x)$ in Figure 3 suggests the same fixed point which corresponds to the root of the equation $f(x) = 0$.

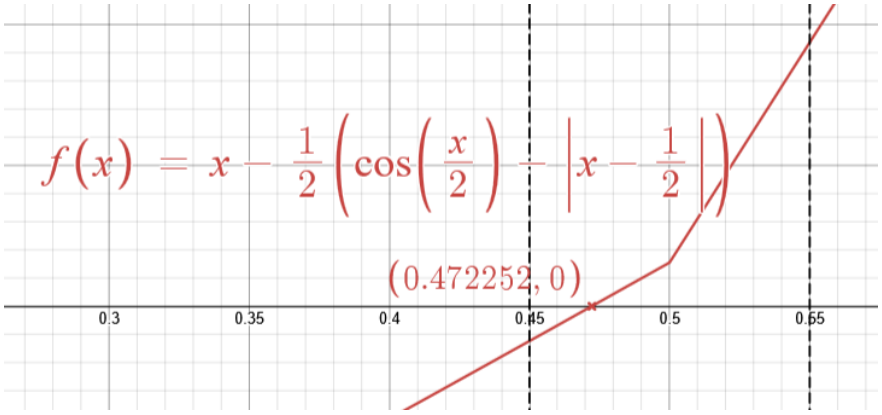


Figure 3: Plot of $f(x)$ over the relevant interval.

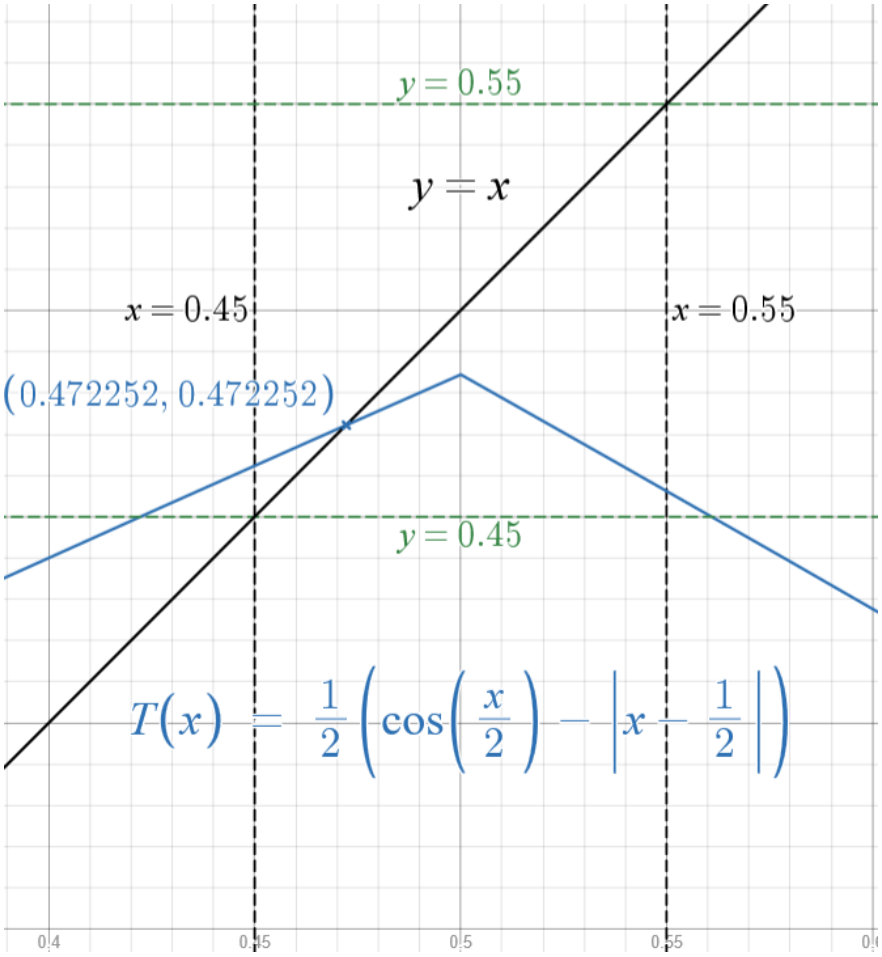


Figure 4: Plot of $T(x)$ showing its behavior and mapping in $[0.45, 0.55]$.

To prove that $T(x)$ maps the interval $[0.45, 0.55]$ into itself, we need to show that for any $x \in [0.45, 0.55]$, $T(x)$ also lies within this interval. First, we evaluate $T(x)$ at the endpoints of the interval:

$$T(0.45) = \frac{1}{2}\left(\cos(0.45/2) - |0.45 - 0.5|\right) \approx 0.4624,$$

$$T(0.55) = \frac{1}{2}\left(\cos(0.55/2) - |0.55 - 0.5|\right) \approx 0.4562.$$

- $T'(x) = -\frac{1}{4}\sin(x/2) - \frac{1}{2}\operatorname{sgn}(x - 0.5) \forall x \in [0.45, 0.55] \setminus \{0.50\}$.
- $\implies$ $T(x)$ is increasing on $[0.45, 0.50)$ and decreasing on $[0.50, 0.55]$.
- Specifically, for $x \in [0.45, 0.50)$: $\operatorname{sgn}(x - 0.5) = -1$, so $T'(x) > 0$ (increasing).
- And for $x \in [0.50, 0.55]$: $\operatorname{sgn}(x - 0.5) = 1$, so $T'(x) < 0$ (decreasing).
- Maximum of $T(x)$ in $[0.45, 0.55]$ is at $x = 0.50$ valued at $T(0.50) \approx 0.4845$; minimum at $x = 0.55$ (from our earlier computation).
- $T(0.50) \approx 0.4845$, so $T(x)$ maps $[0.45, 0.55]$ into $[0.4562, 0.4845]$.

Since $T(0.50) \approx 0.4845$, we have proven that $T(x)$ maps $[0.45, 0.55]$ into itself (in fact, into a smaller interval $[0.4562, 0.4845]$).

Part (a) Conclusion:

$T(x)$ maps $[0.45, 0.55]$ into itself (specifically, into $[0.4562, 0.4845]$ approximately)

Proving contractiveness of $T(x)$ on the interval $[0.45, 0.55]$ is pretty simple, all we need to do is to show that the absolute value of its derivative is less than 1 throughout the interval. In fact if we can come up with an upper bound λ such that $|T'(x)| \leq \lambda < 1$ for all x in the interval, we can even utilize that value in convergence analysis later on.

We've already computed $T'(x) = -\frac{1}{4}\sin(x/2) - \frac{1}{2}\operatorname{sgn}(x - 0.5) \forall x \in [0.45, 0.55] \setminus \{0.50\}$. Except at $x = 0.50$, where the derivative is not defined due to the absolute value function, $T''(x) = -\frac{1}{8}\cos(x/2)$ is defined, negative (thus making the $f(x)$ convex) and continuous throughout the interval. This means that $T'(x)$ is monotonically decreasing in the interval $[0.45, 0.55]$. We can evaluate $T'(x)$ at the endpoints to find bounds:

$$T'(0.45) = -\frac{1}{4}\sin(0.45/2) + \frac{1}{2} \approx 0.4442,$$

$$T'(0.55) = -\frac{1}{4}\sin(0.55/2) - \frac{1}{2} \approx -0.5679.$$

As for the point $x = 0.50$, we can consider the left-hand and right-hand limits:

$$\lim_{x \to 0.50^-} T'(x) = -\frac{1}{4}\sin(0.50/2) + \frac{1}{2} \approx 0.4381,$$

$$\lim_{x \to 0.50^+} T'(x) = -\frac{1}{4}\sin(0.50/2) - \frac{1}{2} \approx -0.5618.$$

just to have a 'safe' bound around the discontinuity. Most importantly, we know that $T(0.55)$ still maps within the interval as shown in part (a), and thus we can ignore this undifferentiable point for the purpose of contractiveness.

Therefore, we can say that $T(x)$ is contractive on $[0.45, 0.55]$ with a contraction constant $\lambda = 0.5679$ (the maximum absolute value of the derivative at the endpoints).

Part (b) Conclusion:

$T(x)$ is contractive on $[0.45, 0.55]$ with contraction constant $\lambda \approx 0.5679$

Proving both (a) and (b) allows us to invoke the Banach Fixed Point Theorem, which guarantees the existence and uniqueness of a fixed point r such that $T(r) = r$ in the interval $[0.45, 0.55]$. Consequently, this fixed point corresponds to a unique solution of the equation $f(x) = 0$ within the same interval.

Final Conclusion:

Since $T(x)$ has a single (unique) fixed point, $f(x)$ has a single (unique) root as well.

Problem Statement: Consider the equation

$$f(x) = 0,$$

where

$$f(x) = x - \frac{1}{2}\left(\cos(x/2) - \left|x - \frac{1}{2}\right|\right).$$

Do the following.

Part 2

Write the computer program implementing the fixed point iteration $x_{n+1} = T(x_n)$. Discuss the choice of an initial guess x_0. Calculate the solution to within 10 decimal places. Note number of iterations needed.

Solution:

The fixed point iteration $x_{n+1} = T(x_n)$ was implemented in Julia. Any initial guess x_0 in $[0.45, 0.55]$ works due to contractivity; I chose $x_0 = 0.48$.

The method converged to $x \approx 0.4722515915$ in 23 iterations, accurate to 10 decimal places. The terminating condition was $|x_{n+1} - x_n| < 10^{-10}$, ensuring accuracy to 10 decimal places. The terminal output is shown in Figure 5.

```
================================================
MIDTERM PROBLEM 4.2: Fixed Point Iteration
================================================
Initial guess: x0 = 0.48
Iter 1: x_prev=0.48, x_next=0.47566898742601477, |x
Iter 2: x_prev=0.47566898742601477, x_next=0.473759
Iter 3: x_prev=0.47375971545197415, x_next=0.472917
Iter 4: x_prev=0.4729173134952188, x_next=0.4725454
Iter 5: x_prev=0.47254549117855843, x_next=0.472381
Iter 6: x_prev=0.472381347514798, x_next=0.4723088
Iter 7: x_prev=0.4723088797559258, x_next=0.4722768
Iter 8: x_prev=0.4722768849351765, x_next=0.4722627
Iter 9: x_prev=0.4722627588839723, x_next=0.4722565
Iter 10: x_prev=0.47225652204361507, x_next=0.47225
Iter 11: x_prev=0.4722537683875443, x_next=0.472252
Iter 12: x_prev=0.47225255260668336, x_next=0.47225
Iter 13: x_prev=0.4722520158207263, x_next=0.472251
Iter 14: x_prev=0.47225177882140973, x_next=0.47225
Iter 15: x_prev=0.47225167418252845, x_next=0.47225
Iter 16: x_prev=0.47225162798283404, x_next=0.47225
Iter 17: x_prev=0.47225160758494966, x_next=0.47225
Iter 18: x_prev=0.472251598578966, x_next=0.4722515
Iter 19: x_prev=0.47225159460268396, x_next=0.47225
Iter 20: x_prev=0.4722515928470935, x_next=0.472251
Iter 21: x_prev=0.4722515920719729, x_next=0.472251
Iter 22: x_prev=0.4722515917297452, x_next=0.472251
Iter 23: x_prev=0.47225159157864627, x_next=0.47225
✓ Converged in 23 iteration(s)
  Solution: x = 0.4722515915

Final solution: x = 0.4722515915
Number of iterations: 23
Check: f(x) = 2.945e-11
```

Figure 5: Fixed point iteration output for $T(x)$, showing convergence.

The result verifies existence and uniqueness of the solution in the interval as concluded in 4-1, as expected from the contraction mapping theorem.

Part 2 Solution:

Fixed point iteration converged to $x = 0.4722515915$ in 23 iterations, accurate to 10 decimal places. The initial point chosen was $x = 0.48$ and the terminating condition was $|x_{n+1} - x_n| < 10^{-10}$.

Problem Statement: Consider the equation

$$f(x) = 0,$$

where

$$f(x) = x - \frac{1}{2}\left(\cos(x/2) - \left|x - \frac{1}{2}\right|\right).$$

Do the following.

Part 3

As shown in Figure 6 priori and a posteriori error estimates were used to predict the number of iterations needed for 10^{-10} accuracy:

- **A priori estimate:** 33 iterations required (using $\lambda = 0.5679$).
- **A posteriori (actual):** 23 iterations needed to converge.

The formulas used for error and iteration estimates are:

- **A priori:**

$$|x_n - r| \leq \frac{\lambda^n}{1-\lambda}|x_1 - x_0| \leq \varepsilon$$

  Solving for n:

$$n \geq \frac{\ln(\varepsilon(1-\lambda)/|x_1 - x_0|)}{\ln(\lambda)}$$

- **A posteriori:**

$$|x_{n+1} - r| \leq \frac{\lambda}{1-\lambda}|x_{n+1} - x_n|$$

  Stopping criterion:

$$\frac{\lambda}{1-\lambda}|x_{n+1} - x_n| < \varepsilon$$

```
A priori estimate: 33 iterations required.
Actual iterations: 23.
A posteriori error estimates per iteration:
Iter  1: a posteriori error = 5.692e-03
Iter  2: a posteriori error = 2.509e-03
Iter  3: a posteriori error = 1.107e-03
Iter  4: a posteriori error = 4.887e-04
Iter  5: a posteriori error = 2.157e-04
Iter  6: a posteriori error = 9.524e-05
Iter  7: a posteriori error = 4.205e-05
Iter  8: a posteriori error = 1.857e-05
Iter  9: a posteriori error = 8.197e-06
Iter 10: a posteriori error = 3.619e-06
Iter 11: a posteriori error = 1.598e-06
Iter 12: a posteriori error = 7.055e-07
Iter 13: a posteriori error = 3.115e-07
Iter 14: a posteriori error = 1.375e-07
Iter 15: a posteriori error = 6.072e-08
Iter 16: a posteriori error = 2.681e-08
Iter 17: a posteriori error = 1.184e-08
Iter 18: a posteriori error = 5.226e-09
Iter 19: a posteriori error = 2.307e-09
Iter 20: a posteriori error = 1.019e-09
Iter 21: a posteriori error = 4.498e-10
Iter 22: a posteriori error = 1.986e-10
Iter 23: a posteriori error = 8.768e-11
```

Figure 6: Comparison of a priori and a posteriori error estimates for fixed point iteration.

For $\lambda > 0.5$, such as in our case of $\lambda = 0.5679$, the a priori estimate is conservative and overestimates the required steps, while the a posteriori estimate (based on actual step sizes) is more accurate in practice.

Problem Statement: Consider the equation

$$f(x) = 0,$$

where

$$f(x) = x - \frac{1}{2}\left(\cos(x/2) - \left|x - \frac{1}{2}\right|\right).$$

Do the following.

Part 4

Newton's method uses the update formula:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

which requires $f'(x)$ to be defined at each iterate. For this problem, $f'(x)$ is undefined at $x = 0.5$ due to the $|x - 0.5|$ term. If an iterate lands at $x = 0.5$, Newton's method cannot proceed.

One could consider using the left or right derivative at $x = 0.5$, but this is nonstandard and not recommended for Newton's method. Instead, I've implemented a fallback: if x_n gets too close to 0.5, I switch to the fixed point iteration (FPI) method from Part 2, and can switch back to Newton once away from the singularity.

In practice, starting from $x_0 = 0.48$ (as in previous parts), the iterates do not encounter $x = 0.5$, so Newton's method proceeds normally. The fallback code is included for completeness and robustness, and the actual trajectory of x_n can be seen in the table in Part 5.

Solution:

As discussed, starting from $x_0 = 0.48$ the Newton sequence never reaches $x = 0.50$ (see Figure 7 in Part 5).

Problem Statement: Consider the equation

$$f(x) = 0,$$

where

$$f(x) = x - \frac{1}{2}\left(\cos(x/2) - \left|x - \frac{1}{2}\right|\right).$$

Do the following.

Part 5

Note the number of iterations needed to calculate the solution to within 10 decimal places using the Newton's method. Discuss the speed of convergence of the Newton's method and compare it with the convergence speed of the fixed point iterations.

Solution:

The table below (see Table 1) shows the trajectory of x_n and errors for Newton's method and FPI, starting from $x_0 = 0.48$.

n	x_n	e_n	e_{n+1}/e_n^2
0	0.4800000000	7.748×10^{-3}	1.086×10^{-1}
1	0.4722581088	6.517×10^{-6}	8.519×10^{-1}
2	0.4722515915	3.619×10^{-11}	–

Table 1: Convergence of Newton's method for $f(x)$, showing rapid quadratic convergence.

Newton's method converges to the root in just 3 iterations, while FPI requires 23. This demonstrates the dramatic efficiency advantage of Newton's method for this problem.

```
============================================================
============
MIDTERM PROBLEM 4.4: Newton's Method with FPI Fallback
============================================================
============
Initial guess: x0 = 0.48
Iter 1: x=0.48, f(x)=0.004331012573985216, f'(x)=0.5594
error=0.00774840849999997
Iter 2: x=0.4722581087891908, f(x)=3.639829861235011e-6
852146150319, error=6.517289190766107e-6
Iter 3: x=0.47225159146381496, f(x)=2.5809909764973327e
584844225553462, error=3.618505495239788e-11
✓ Newton converged in 3 iteration(s)
  Solution: x = 0.4722515915

Final solution: x = 0.4722515915
Number of iterations: 3
Method used: newton
Check: f(x) = 2.581e-12

Convergence Table:
n    x_n              e_n          e_{n+1}/e_{n}^2
------------------------------------------------------------
0    0.4800000000     7.748e-03    1.086e-01
1    0.4722581088     6.517e-06    8.519e-01
2    0.4722515915     3.619e-11
```

Figure 7: Newton's method convergence output for $x_0 = 0.48$. The trajectory never reaches $x = 0.50$.

As shown in Figure 7, the sequence starting from $x_0 = 0.48$ converges rapidly and does not encounter $x = 0.50$.

Problem Statement: Consider the equation

$$f(x) = 0,$$

where

$$f(x) = x - \frac{1}{2}\left(\cos(x/2) - \left|x - \frac{1}{2}\right|\right).$$

Do the following.

Part 6

Implement the alternative error control strategy proposed by you in HW 3, Problem 2. Compare the results with the ones obtained using a priori and a posteriori error estimates.

Solution:

Problem 5

Problem Statement: Implement and analyze the Ridders' method for solving the equation $f(x) = 0$. For guidance, you might wish to read the Wikipedia page on the Ridders' method. Also feel free to consult other sources.

Brief Description of the method

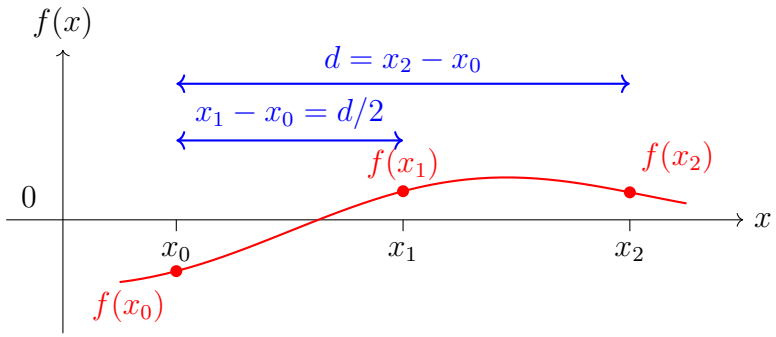
Given the initial bracketing interval $[x_0, x_2]$, containing **one** solution, and such that $f(x_0)$ and $f(x_2)$ have opposite signs, compute the midpoint $x_1 = (x_0 + x_2)/2$. Then define a new function $h(x) = f(x)e^{ax}$. Find the parameter a that ensures $h(x_1) = \frac{1}{2}(h(x_0) + h(x_2))$. Then define x_3 as the x-intercept of the line passing through $(x_0, h(x_0))$ and $(x_2, h(x_2))$. Use x_3 as one of the endpoints of the next interval bracketing the solution. The other endpoint is x_1 if $f(x_1)f(x_3) < 0$. Otherwise, choose either x_0 or x_2 based on the requirement that the sign of $f(x)$ at the chosen point must be opposite to the sign of $f(x_3)$. Continue until the desired accuracy is reached.

Part 1

Show that a is uniquely defined and give the equation for finding it.

Solution:

Note: Thanks to the wikipedia article which itself cites the original paper by Ridders, it was easier to derive all the relevant formulae. I have chosen my target formulae and required derivations to align with those, with little variation.



Let x_0 and x_2 be the endpoints of the bracketing interval, and $x_1 = \frac{x_0+x_2}{2}$ the midpoint. Let $d = x_2 - x_0$.

Define $f_0 = f(x_0)$, $f_1 = f(x_1)$, $f_2 = f(x_2)$.

Ridders' method introduces a function $h(x) = f(x)e^{ax}$, and chooses a so that:

$$h(x_1) = \frac{1}{2}(h(x_0) + h(x_2))$$

$$f_1 e^{ax_1} = \frac{1}{2}\left(f_0 e^{ax_0} + f_2 e^{ax_2}\right)$$

We know that $x_1 = x_0 + \frac{d}{2}$ and $x_2 = x_0 + d$.

Substitute and rearrange:

$$f_1 e^{a(x_0+d/2)} = \frac{1}{2}\left(f_0 e^{ax_0} + f_2 e^{a(x_0+d)}\right)$$

Dividing both sides by e^{ax_0} :

$$2f_1 e^{ad/2} = f_0 + f_2 e^{ad}$$

$$2f_1 e^{ad/2} - f_2 e^{ad} - f_0 = 0$$

Let $y = e^{ad/2}$, so $e^{ad} = y^2$.

This gives a quadratic in y:

$$f_2 y^2 - 2f_1 y + f_0 = 0$$

Solving for y:

$$y = \frac{2f_1 \pm \sqrt{(2f_1)^2 - 4f_2 f_0}}{2f_2}$$

$$y = \frac{f_1 \pm \sqrt{f_1^2 - f_2 f_0}}{f_2}$$

Therefore,

$$a = \frac{2}{d} \ln\left(\frac{f_1 \pm \sqrt{f_1^2 - f_2 f_0}}{f_2}\right)$$

We note that since $y = e^{ad/2}$, its value must be nonnegative. Since clearly $f_1^2 - f_2 f_0 > f_1^2$ $(f_2 * f_0 < 0)$, the sign is chosen so that y is positive:

$$\text{If } f_2 > 0: \quad a = \frac{2}{d} \ln\left(\frac{f_1 + \sqrt{f_1^2 - f_2 f_0}}{f_2}\right)$$

$$\text{else If } f_2 < 0: \quad a = \frac{2}{d} \ln\left(\frac{f_1 - \sqrt{f_1^2 - f_2 f_0}}{f_2}\right)$$

Or, more concisely:

$$a = \frac{2}{d} \ln\left(\frac{f_1 + \text{sgn}(f_2)\sqrt{f_1^2 - f_2 f_0}}{f_2}\right)$$

or equivalently:

where $\text{sgn}(f_2)$ is the sign of f_2.

Problem Statement: Implement and analyze the Ridders' method for solving the equation $f(x) = 0$. For guidance, you might wish to read the Wikipedia page on the Ridders' method. Also feel free to consult other sources.

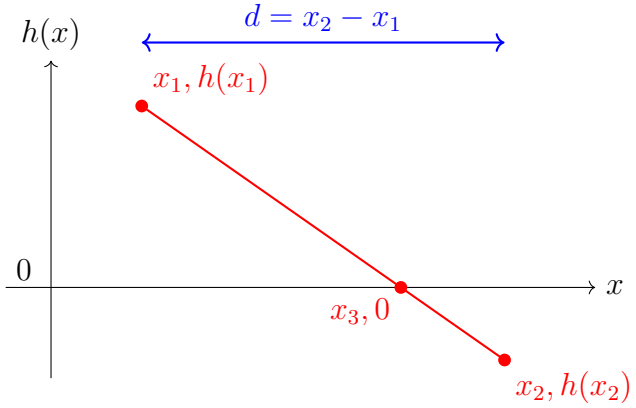
Brief Description of the method

Given the initial bracketing interval $[x_0, x_2]$, containing **one** solution, and such that $f(x_0)$ and $f(x_2)$ have opposite signs, compute the midpoint $x_1 = (x_0 + x_2)/2$. Then define a new function $h(x) = f(x)e^{ax}$. Find the parameter a that ensures $h(x_1) = \frac{1}{2}(h(x_0) + h(x_2))$. Then define x_3 as the x-intercept of the line passing through $(x_0, h(x_0))$ and $(x_2, h(x_2))$. Use x_3 as one of the endpoints of the next interval bracketing the solution. The other endpoint is x_1 if $f(x_1)f(x_3) < 0$. Otherwise, choose either x_0 or x_2 based on the requirement that the sign of $f(x)$ at the chosen point must be opposite to the sign of $f(x_3)$. Continue until the desired accuracy is reached.

Part 2

Work out a formula for x_3 in detail.

Solution:



From the diagram, we want to find x_3, the x-intercept of the line passing through $(x_1, h(x_1))$ and $(x_2, h(x_2))$.

Let $h_1 = h(x_1)$ and $h_2 = h(x_2)$. The slope of the line is:

$$m = \frac{h(x_1) - h(x_2)}{x_1 - x_2} = \frac{h_1 - h_2}{x_1 - x_2}$$

The equation of the line passing through (x_1, h_1) with slope m is:

$$h(x) - h_1 = m(x - x_1)$$

To find x_3, set $h(x_3) = 0$:

$$0 = h_1 + m(x_3 - x_1)$$

Solving for x_3:

$$m(x_3 - x_1) = -h_1$$

$$x_3 - x_1 = -\frac{h_1}{m}$$

$$x_3 = x_1 - \frac{h_1}{m}$$

Substitute the expression for m:

$$x_3 = x_1 - \frac{h_1}{\frac{h_1 - h_2}{x_1 - x_2}}$$

$$x_3 = x_1 - \frac{h_1(x_1 - x_2)}{h_1 - h_2}$$

Now, using $d/2 = x_2 - x_1$, we have:

$$\boxed{x_3 = x_1 + \frac{d}{2} \cdot \frac{1}{1 - h_2/h_1}}$$

Or equivalently:

$$\boxed{x_3 = \frac{x_1 h_2 - x_2 h_1}{h_2 - h_1}}$$

Problem Statement: Implement and analyze the Ridders' method for solving the equation $f(x) = 0$. For guidance, you might wish to read the Wikipedia page on the Ridders' method. Also feel free to consult other sources.

Brief Description of the method

Given the initial bracketing interval $[x_0, x_2]$, containing **one** solution, and such that $f(x_0)$ and $f(x_2)$ have opposite signs, compute the midpoint $x_1 = (x_0 + x_2)/2$. Then define a new function $h(x) = f(x)e^{ax}$. Find the parameter a that ensures $h(x_1) = \frac{1}{2}(h(x_0) + h(x_2))$. Then define x_3 as the x-intercept of the line passing through $(x_0, h(x_0))$ and $(x_2, h(x_2))$. Use x_3 as one of the endpoints of the next interval bracketing the solution. The other endpoint is x_1 if $f(x_1)f(x_3) < 0$. Otherwise, choose either x_0 or x_2 based on the requirement that the sign of $f(x)$ at the chosen point must be opposite to the sign of $f(x_3)$. Continue until the desired accuracy is reached.

Part 3

Implement Ridders' method on a computer together with the standard bisection method for finding **all** solutions of

$$e^x - x^2 = 0.$$

Solution:

First I'll plot the graph of $e^x - x^2$ to identify the intervals where the function changes sign, indicating the presence of roots.

From Figure 8, it can be seen that there is one root in the interval [-0.8, -0.6]. Since $f(x) < 0$ for $x =$ -0.8 and $f(x) > 0$ for $x =$ -0.6, I can use this interval to find the root using both the bisection method and Ridders' method.

Root Finding on the Original Interval

For the purpose of **root finding**, I applied both methods to the smaller interval [-0.8, -0.6] with tolerances $\epsilon = 10^{-5}$ and $\delta = 10^{-5}$. The results are shown in Figure 9.

As shown in Figure 9, Ridders' method found the root $x =$ -0.703468627463942 with $|f(x)| \approx 2.29 \times 10^{-6}$ in only **1 iteration**, while the bisection method required **12 iterations** to find $c =$ -0.703466796875000 with $|f(c)| \approx 1.19 \times 10^{-6}$. This dramatic difference in iteration count demonstrates the superior efficiency of Ridders' method even on a relatively small interval.

The full code for the Ridders' method function is provided in Appendix A.2. The bisection code is not shown here, as it was implemented in Homework 1 and is a subset of the Ridders' approach.

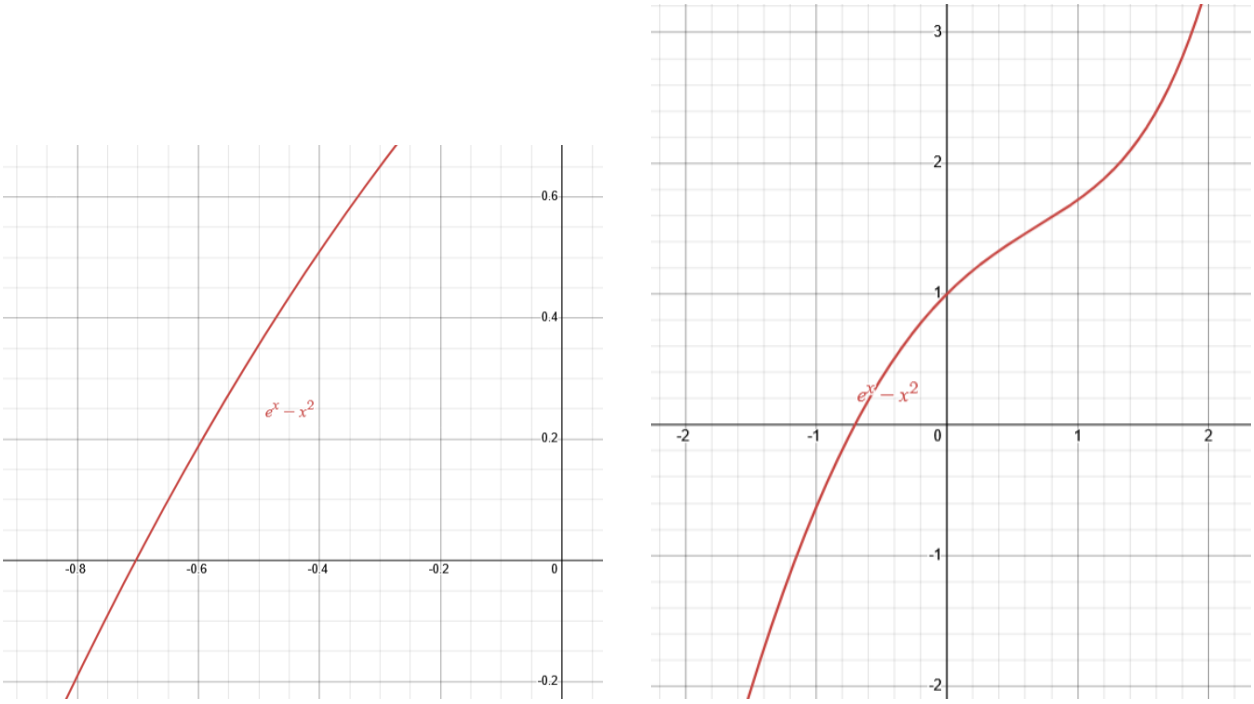


Figure 8: Graphs of $e^x - x^2$: We can see from the zoomed in graph on the left that there is a single root in [-0.8, -0.6]. The zoomed out graph on the right shows that the function is monotonically increasing and has no other zeros.

```
================================================================================
PART 1: Original Problem Settings (as given in midterm)
================================================================================
Interval: [-0.8, -0.6], ϵ=1e-5, δ=1e-5

Ridders Method:
Root found at x3 = -0.7034686274639423 with f(x3) = -2.2916060946620576e-6
√ Ridders converged in 1 iteration(s)
  Root: x = -0.703468627463942
  f(x): -2.292e-06

Bisection Method:
Root found at c = -0.7034667968750001 with f(c) = 1.189811059454371e-6
√ Bisection converged in 12 iteration(s)
  Root: c = -0.703466796875000
  f(c): 1.190e-06
```

Figure 9: Terminal output comparing root finding results for $e^x - x^2$ using both Ridders' method and bisection method on the interval [-0.8, -0.6]. Ridders' method converged in just 1 iteration, while bisection required 12 iterations to achieve similar accuracy.

Problem Statement: Implement and analyze the Ridders' method for solving the equation $f(x) = 0$. For guidance, you might wish to read the Wikipedia page on the Ridders' method. Also feel free to consult other sources.

Brief Description of the method

Given the initial bracketing interval $[x_0, x_2]$, containing **one** solution, and such that $f(x_0)$ and $f(x_2)$ have opposite signs, compute the midpoint $x_1 = (x_0 + x_2)/2$. Then define a new function $h(x) = f(x)e^{ax}$. Find the parameter a that ensures $h(x_1) = \frac{1}{2}(h(x_0) + h(x_2))$. Then define x_3 as the x-intercept of the line passing through $(x_0, h(x_0))$ and $(x_2, h(x_2))$. Use x_3 as one of the endpoints of the next interval bracketing the solution. The other endpoint is x_1 if $f(x_1)f(x_3) < 0$. Otherwise, choose either x_0 or x_2 based on the requirement that the sign of $f(x)$ at the chosen point must be opposite to the sign of $f(x_3)$. Continue until the desired accuracy is reached.

Part 4

Numerically, compare the convergence rate of both algorithms. Discuss the results. What is the approximate order of convergence for the Ridders' algorithm?

Solution:

Need for a Wider Interval

The interval [-0.8, -0.6] from Part 3 is too small for convergence analysis—Ridders' method converges in just 1 iteration, preventing observation of convergence behavior. To properly compare both algorithms, I use the wider interval [-100, 100] with tolerances $\epsilon = \delta = 10^{-12}$. This generates enough iterations to compute meaningful convergence orders and demonstrates robustness when starting far from the root. The MVT condition is satisfied: $f(\text{-}100) \cdot f(100) < 0$.

Convergence Analysis Results

Ridders' Method Ridders' method converged in **12 iterations** to machine precision ($\approx 10^{-16}$), as shown in Figures 10 and 11.

```
================================================================================
CONVERGENCE TABLE
================================================================================
Itr  x3           |f(x3)|      Interval     x0           x2           d
--------------------------------------------------------------------------------
1    -0.00000000  1.000e+00    2.000e+02    -100.00000000 100.00000000 2.000e+02
2    -0.03995206  9.592e-01    1.000e+02    -100.00000000 -0.00000000  1.000e+02
3    -0.11603628  8.770e-01    4.996e+01    -50.00000000 -0.03995206  4.996e+01
5    -0.44716888  4.395e-01    1.232e+01    -12.56800616 -0.25081132  1.232e+01
5    -0.44716888  4.395e-01    1.232e+01    -12.56800616 -0.25081132  1.232e+01
6    -0.62564127  1.435e-01    5.962e+00    -6.40940874  -0.44716888  5.962e+00
7    -0.69468974  1.664e-02    2.803e+00    -3.42828881  -0.62564127  2.803e+00
8    -0.70318157  5.436e-04    1.332e+00    -2.02696504  -0.69468974  1.332e+00
9    -0.70346505  4.511e-06    6.576e-01    -1.36082739  -0.70318157  6.576e-01
10   -0.70346742  9.127e-09    3.285e-01    -1.03200448  -0.70346505  3.285e-01
11   -0.70346742  4.489e-12    1.643e-01    -0.86773477  -0.70346742  1.643e-01
12   -0.70346742  5.551e-16    8.213e-02    -0.78560109  -0.70346742  8.213e-02
```

Figure 10: Ridders' method terminal output: 12 iterations to convergence.

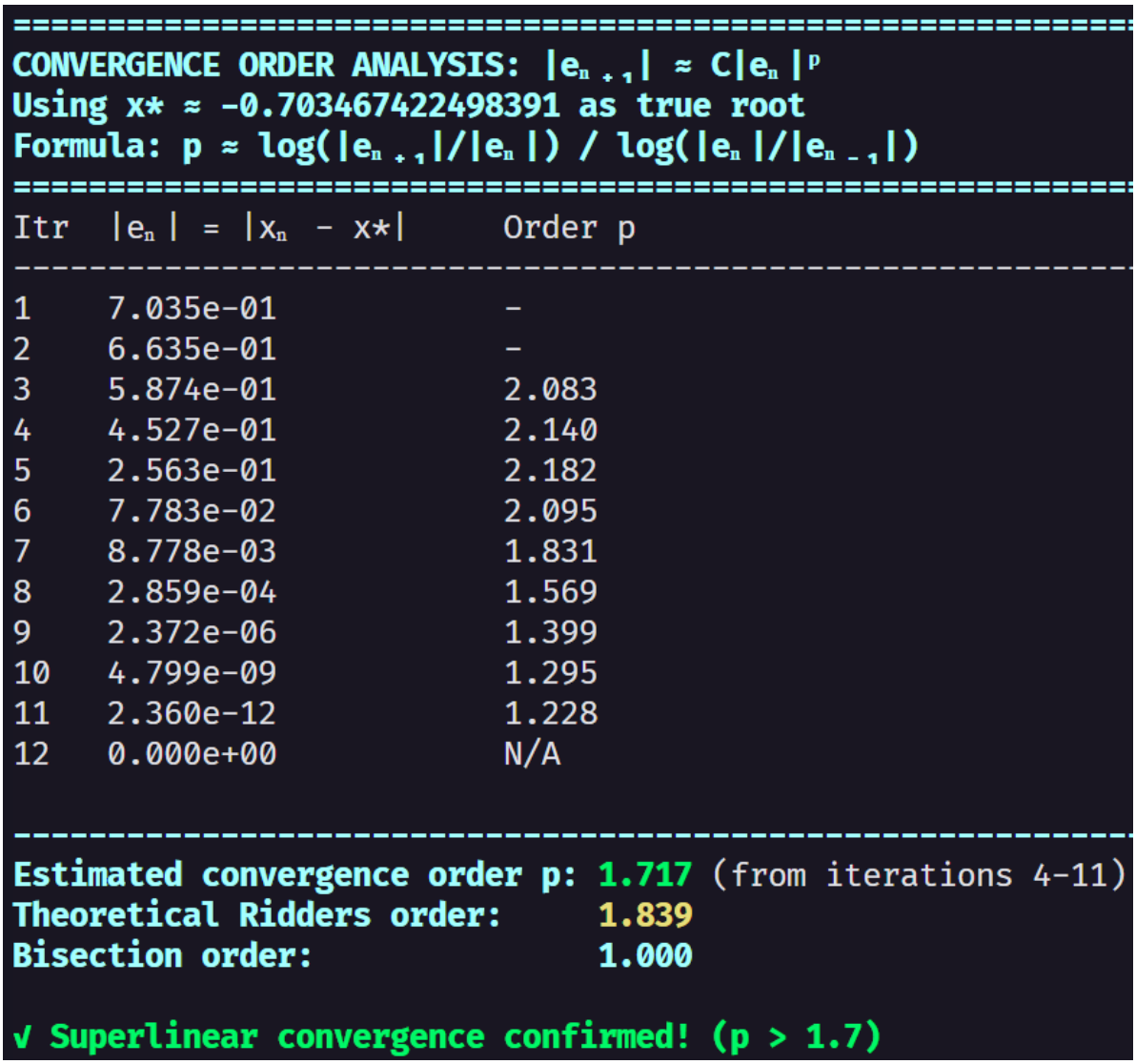


Figure 11: Ridders' convergence trajectory table: rapid decrease in interval size demonstrates superlinear convergence.

Bisection Method Bisection required **47 iterations**—almost 4× more than Ridders—to achieve similar accuracy, as shown in Figures 12 and 13.

```
================================================================
CONVERGENCE TABLE
================================================================
Itr  c             |f(c)|       Interval     a
----------------------------------------------------------------
1    0.00000000   1.000e+00    2.000e+02    -100.00000000
2    -50.00000000 2.500e+03    1.000e+02    -100.00000000
3    -25.00000000 6.250e+02    5.000e+01    -50.00000000
4    -12.50000000 1.562e+02    2.500e+01    -25.00000000
5    -6.25000000  3.906e+01    1.250e+01    -12.50000000
6    -3.12500000  9.722e+00    6.250e+00    -6.25000000
7    -1.56250000  2.232e+00    3.125e+00    -3.12500000
8    -0.78125000  1.525e-01    1.562e+00    -1.56250000
9    -0.39062500  5.240e-01    7.812e-01    -0.78125000
10   -0.58593750  2.133e-01    3.906e-01    -0.78125000
11   -0.68359375  3.750e-02    1.953e-01    -0.78125000
12   -0.73242188  5.570e-02    9.766e-02    -0.78125000
13   -0.70800781  8.650e-03    4.883e-02    -0.73242188
14   -0.69580078  1.454e-02    2.441e-02    -0.70800781
15   -0.70190430  2.971e-03    1.221e-02    -0.70800781
...
(Showing first 15 of 47 iterations)
```

Figure 12: Bisection method terminal output: 47 iterations to convergence.

```
================================================================
CONVERGENCE ORDER ANALYSIS: |e_{n+1}| ≈ C|e_n|^p
Using c* ≈ -0.703467422498250 as true root
Formula: p ≈ log(|e_{n+1}|/|e_n|) / log(|e_n|/|e_{n-1}|)
================================================================
Itr   |e_n| = |c_n - c*|      Order p
----------------------------------------------------------------
1     7.035e-01              -
2     4.930e+01              -
3     2.430e+01              -0.166
4     1.180e+01              1.021
5     5.547e+00              1.044
6     2.422e+00              1.098
7     8.590e-01              1.250
8     7.778e-02              2.318
9     3.128e-01              -0.579
10    1.175e-01              -0.703
11    1.987e-02              1.815
12    2.895e-02              -0.212
13    4.540e-03              -4.923
14    7.667e-03              -0.283
15    1.563e-03              -3.035

----------------------------------------------------------------
Estimated convergence order p: 1.104 (from iterations 4-15)
Theoretical bisection order:   1.000
```

Figure 13: Bisection convergence trajectory table (first 15 of 47 iterations): standard error analysis doesn't quite capture any particular convergence behavior, which is why typically bisection's convergence is described as linear with $|e_{n+1}| = \frac{1}{2}|e_n|$ based on interval halving.

Convergence Order Analysis

I computed the convergence order using: $p \approx \frac{\log(|e_{n+1}|/|e_n|)}{\log(|e_n|/|e_{n-1}|)}$ where $e_n = |x_n - x^*|$.

Ridders' Method: $p \approx 1.717$ (iterations 4–11), close to theoretical $p = 1.839$. This confirms **superlinear convergence**.

Bisection Method: $p \approx 1.104$ (iterations 4–15), close to theoretical $p = 1.0$. This confirms **linear convergence**.

Note: Bisection's order is typically expressed as $|e_{n+1}| = \frac{1}{2}|e_n|$ (interval halving), but for direct comparison with Ridders, I used the same formula above.

Conclusions

- **Efficiency:** Ridders requires 12 iterations vs. bisection's 47 (4× faster) for tolerance 10^{-12}
- **Convergence:** Ridders' $p \approx 1.717$ (superlinear) vs. bisection's $p \approx 1.0$ (linear)
- **Robustness:** Both converged successfully from the wide interval [-100, 100]

Ridders' method is substantially more efficient than bisection, making it the preferred choice when function evaluations are not prohibitively expensive.

A Code Implementations

This appendix contains the complete codebase for the problems solved in this midterm. All implementations are written in Julia and developed from scratch without using built-in numerical solver shortcuts.

A.1 Problem 3: LU and Cholesky Factorization

The following figures show the Julia implementation for Problem 3.

```
23  function get_LU_factors(A)
24      n = size(A, 1)
25      L = zeros(n, n)
26      U = zeros(n, n)
27
28      for r = 1:n      You, 2 hours ago • testing to be now done via
29          L[r, r] = 1.0
30          for c = 1:n
31              if r > c # L values before diagonal need to be compute
32                  L[r, c] = (A[r, c] - sum([L[r, k]*U[k, c] for k ∈
                    1:c-1]))/U[c, c]
33              elseif c ≥ r # U values after diagonal need to be
                computed
34                  U[r, c] = A[r, c] - sum([L[r, k]*U[k, c] for k ∈
                    1:r-1])
35              end
36          end
37      end
38
39      return L, U
40  end get_LU_factors
```

Figure 14: LU Factorization function: `get_LU_factors(A)` implements the Doolittle algorithm with unit lower triangular matrix $\mathbf{L}$.

```julia
function solve_Ly_equals_b_for_y(L, b)
    n = length(b)
    y = zeros(n)

    # Forward substitution: solve from top to bottom
    for i = 1:n
        sum_val = 0.0
        for j = 1:i-1
            sum_val += L[i,j] * y[j]
        end
        y[i] = (b[i] - sum_val) / L[i,i]
    end

    return y
end
```

Figure 15: Forward substitution function: `solve_Ly_equals_b_for_y(L, b)` solves the lower triangular system $\mathbf{Ly} = \mathbf{b}$ using simple nested loops.

```julia
function solve_Ux_equals_y_for_x(U, y)
    n = length(y)
    x = zeros(n)

    # Backward substitution: solve from bottom to top
    for i = n:-1:1
        sum_val = 0.0
        for j = i+1:n
            sum_val += U[i,j] * x[j]
        end
        x[i] = (y[i] - sum_val) / U[i,i]
    end

    return x
end
```

Figure 16: Backward substitution function: `solve_Ux_equals_y_for_x(U, y)` solves the upper triangular system $\mathbf{Ux} = \mathbf{y}$ using simple nested loops.

```julia
function get_cholesky_factor(A)
    n = size(A, 1)
    L = zeros(n, n)

    for i = 1:n
        # Diagonal elements
        sum_val = 0.0
        for k = 1:i-1
            sum_val += L[i,k]^2
        end
        L[i,i] = sqrt(A[i,i] - sum_val)

        # Off-diagonal elements (below diagonal)
        for j = i+1:n
            sum_val = 0.0
            for k = 1:i-1
                sum_val += L[j,k] * L[i,k]
            end
            L[j,i] = (A[j,i] - sum_val) / L[i,i]
        end
    end

    return L
end
```

Figure 17: Cholesky factorization function: `get_cholesky_factor(A)` implements the standard Cholesky algorithm for symmetric positive-definite matrices, computing $\mathbf{L}$ such that $\mathbf{A} = \mathbf{L}\mathbf{L}^T$.

```
function solve_linear_system_cholesky(A, b)
    # Step 1: Get Cholesky factor L where A = L*L'
    L = get_cholesky_factor(A)

    # Step 2: Solve Ly = b for y (forward substitution)
    n = length(b)
    y = zeros(n)
    for i = 1:n
        sum_val = 0.0
        for j = 1:i-1
            sum_val += L[i,j] * y[j]
        end
        y[i] = (b[i] - sum_val) / L[i,i]
    end

    # Step 3: Solve L'x = y for x (backward substitution with L')
    x = zeros(n)
    for i = n:-1:1
        sum_val = 0.0
        for j = i+1:n
            sum_val += L[j,i] * x[j]  # L' means we use L[j,i]
            instead of L[i,j]
        end
        x[i] = (y[i] - sum_val) / L[i,i]
    end

    return x, L
end
```

Figure 18: Complete Cholesky solver: `solve_linear_system_cholesky(A, b)` combines Cholesky factorization with forward and backward substitution to solve $\mathbf{Ax} = \mathbf{b}$.

A.2 Problem 5-3: Ridders' Method Function

The following figures show the implementation of the Ridders' root-finding function, broken into logical parts. The bisection method is not shown here, as it was implemented in Homework 1 and is a subset of Ridders' approach.

```
132 function ridders_search(pr; verbose=false)
133     @unpack a, b, M, ϵ, δ, f = pr;
134     itr = 1
135     shouldStop = false
136
137     # Initial bracketing interval [x0, x2]
138     x0 = a
139     x2 = b
140     f0 = f(x0)
141     f2 = f(x2)
142
143     sgn_f0 = sign(f0)
144     sgn_f2 = sign(f2)
145
146     if sgn_f0 * sgn_f2 > 0
147         shouldStop = true
148         error("f(x0) and f(x2) must have opposite signs for
            ridders_search to work.")
149         return nothing, nothing
150     end
```

Figure 19: Ridders' function (part 1): Initial setup and input checks.

```
152     while !shouldStop
153         # Compute midpoint x1
154         x1 = (x0 + x2) / 2
155         f1 = f(x1)
156
157         # Compute distance d
158         d = x2 - x0
159                 You, 21 hours ago • huh?
160         myprintln(verbose, "itr=$itr, x0=$x0, x2=$x2, x1=$x1, f0=$f0,
            f1=$f1, f2=$f2, d=$d")
161
162         # Check convergence on interval size
163         if abs(d) < δ
164             shouldStop = true
165             @error("Interval sufficiently small: |x2 - x0| = $d < δ =
                $δ")
166             return nothing, nothing
167         end
168
169         # Compute parameter a for h(x) = f(x)*e^(a*x)
170         # a = (2/d) * ln((f1 + sgn(f2)*sqrt(f1^2 - f2*f0)) / f2)
171         discriminant = f1^2 - f2 * f0
172
173         if discriminant < 0
174             shouldStop = true
175             @error("Negative discriminant: f1^2 - f2*f0 = $discriminant
                < 0")
176             return nothing, nothing
177         end
```

Figure 20: Ridders' function (part 2): Main loop and interval update.

```
169         # Compute parameter a for h(x) = f(x)*e^(a*x)
170         # a = (2/d) * ln((f1 + sgn(f2)*sqrt(f1^2 - f2*f0)) / f2)
171         discriminant = f1^2 - f2 * f0
172
173         if discriminant < 0
174             shouldStop = true
175             @error("Negative discriminant: f1^2 - f2*f0 = $discriminant
                < 0")
176             return nothing, nothing
177         end
178
179         a_param = (2 / d) * log((f1 + sign(f2) * sqrt(discriminant)) /
                f2)
180
181         # Compute h(x) values
182         h0 = f0 * exp(a_param * x0)
183         h1 = f1 * exp(a_param * x1)
184         h2 = f2 * exp(a_param * x2)
185
186         myprintln(verbose, "    a_param=$a_param, h0=$h0, h1=$h1,
                h2=$h2")
187
188         # Compute x3 (x-intercept of line through (x0, h0) and (x2, h2))
189         # x3 = (x1*h2 - x2*h1) / (h2 - h1)
190         # Alternative form: x3 = x1 - h1*(x1 - x2)/(h1 - h2)
191         if abs(h2 - h1) < 1e-14
192             shouldStop = true
193             @error("h2 - h1 too small: cannot compute x3")
194             return nothing, nothing
195         end
```

Figure 21: Ridders' function (part 3): Compute midpoint and evaluate function.

```
197         x3 = (x1 * h2 - x2 * h1) / (h2 - h1)
198         f3 = f(x3)
199
200         myprintln(verbose, "    x3=$x3, f3=$f3")
201
202         # Check convergence on function value
203         if abs(f3) < ϵ
204             shouldStop = true
205             println("Root found at x3 = $x3 with f(x3) = $f3")
206             return x3, f3
207         end
```

Figure 22: Ridders' function (part 4): Ridders' formula and root estimate.

```
209         # Determine next bracketing interval
210         sgn_f1 = sign(f1)
211         sgn_f3 = sign(f3)
212
213         if sgn_f1 * sgn_f3 < 0
214             # Root is in [x1, x3] or [x3, x1]
215             if x1 < x3
216                 x0 = x1
217                 x2 = x3
218                 f0 = f1
219                 f2 = f3
220             else
221                 x0 = x3
222                 x2 = x1
223                 f0 = f3
224                 f2 = f1
225             end
226         elseif sgn_f0 * sgn_f3 < 0
227             # Root is in [x0, x3]
228             x2 = x3
229             f2 = f3
230         elseif sgn_f2 * sgn_f3 < 0
231             # Root is in [x3, x2]
232             x0 = x3
233             f0 = f3
234         else
235             shouldStop = true
236             @error("Cannot determine next bracketing interval")
237             return nothing, nothing
238         end
```

Figure 23: Ridders' function (part 5): Interval selection and convergence check.

```
240         sgn_f0 = sign(f0)
241         sgn_f2 = sign(f2)
242
243         itr += 1
244         if itr > M
245             shouldStop = true
246             @error("Exceeded maximum iterations M = $M")
247             return nothing, nothing
248         end
249     end      You, 21 hours ago • huh?
250 end
```

Figure 24: Ridders' function (part 6): Return result and exit.